Technical University of Munich

Exercises Rechnereinführung WS 2026/27

Adrian Stein – Thomas Lindae – Ari Breitkreuz

---

1 – Linux (Part 1)

Exercises marked with a (*) are bonus tasks and should only be done if there is enough time left.

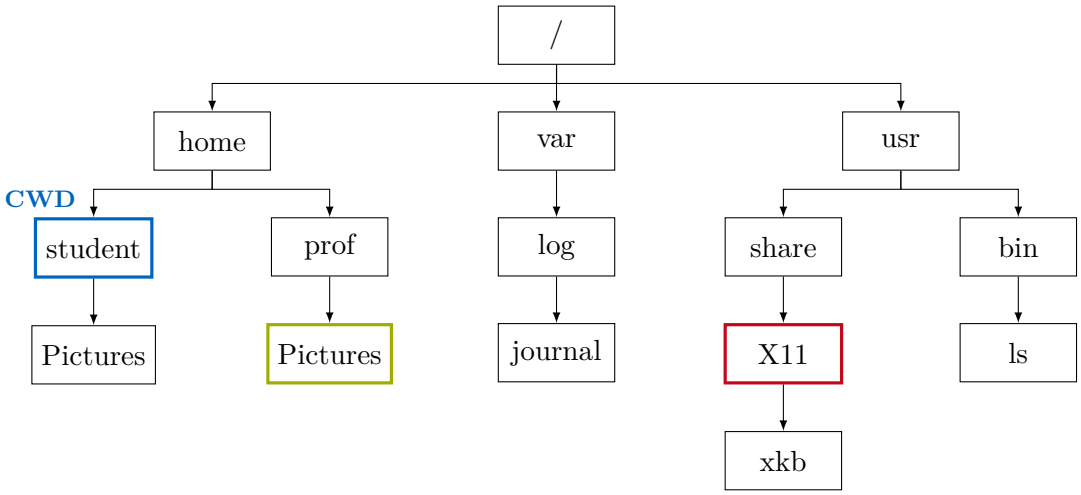
1.1 Important

As already discussed in the lecture, the goal is not that you know every command by heart with all possible options. On this sheet we want to practice using the terminal to internalize the most important commands to some extent. You will also learn how you can help yourself by using e.g. the Linux manual (through the command `man`).

1.2 The Linux-Filesystem

1.2.1 Filesystem Paths

Assume the following snippet of a filesystem in Linux:



Give a path to the green and red folders respectively as a(n):

1. absolute path
2. relative path
3. path containing a `~` (assume you are logged in as the user "prof")

Solution

Green:

1. `/home/prof/Pictures`
2. `../prof/Pictures`
3. `~/Pictures`

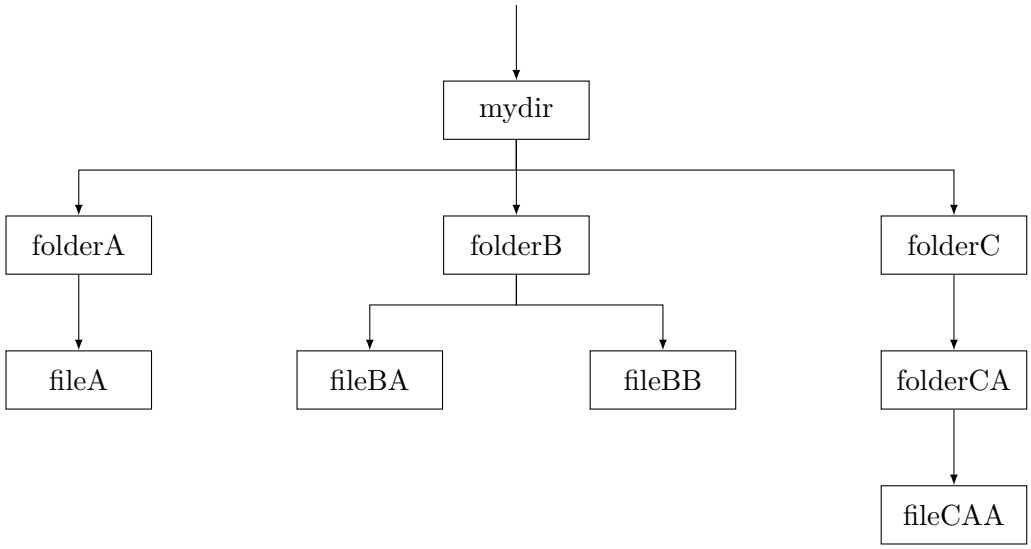
Red:

1. `/usr/share/X11`
2. `../../usr/share/X11`
3. `~/../../usr/share/X11`

Mind the case sensitivity!

1.2.2 Create Folders with `mkdir`

1. In a folder of your choice, use `mkdir` and `touch` to create the following structure:



2. Read through the `man` page of `mkdir`. How could you create the folder `~/uni/vorkurs/rechner` with **exactly one** command?
3. (*) Use the same command as before, but also print the created folders to the command line. Only use a single `-` in your command.

Solution

1. Easy to verify with the command `tree`.
2. `mkdir -p Documents/uni/vorkurs` (`-p` is the decisive argument)
3. `mkdir -pv Documents/uni/vorkurs`. The use of `--parent --verbose` would be equivalent, but only one `-` should be used.

1.2.3 Navigation

Use `cd` with different targets a few times. Try to figure out what the command `cd -` does.

Solution

`cd -` brings you back to the previous folder.

1.2.4 Listing Directories and Hidden Files with `ls`

Read through the `man` page for `ls`.

1. How are files "hidden" in linux? How can you list such files?
2. Is `..` displayed in the root directory? How can you find this out without leaving your current directory?
3. List **all** files in your current directory (with all information), except for `.` and `..`, color the names of directories and sort the files by newest access time.

Solution

1. "Hidden" files start with a dot. You can list them with `ls -a`
2. Yes, with the command `ls -a /`
3. `ls -Altu --color=auto` or `ls -A -l -t -u --color=auto` (or any permutation of these flags)

1.2.5 (*) Timestamps and `touch`

Read through the `man` page for `touch`.

1. Create a file `very.old` which has the modification date 01.01.1990. Use `ls` to check your result.
2. Change the modification date of the file to the current time.

Solution

1. `touch -t 199001010000 very.old`
2. `touch -m very.old`

1.2.6 Deletions with `rm`

Read through the `man` page for `rm`.

1. Is it possible to recover a file that was deleted with `rm`?
2. What is the difference between `shred` and `rm`? Can files deleted with `shred` be recovered?
3. (*) On the internet, a joke exists about using the command `rm -rf /` to delete the entire operating system. What protection does `rm` offer to prevent the accidental use of such a command?

Solution

1. There is no "Recycle Bin" from which one could explicitly recover a file. Nevertheless, files are not erased completely from the physical medium. One could recover a file with enough effort.

   A simple demo for *native* Linux:

   To illustrate this, it is best to create a minimal partition (at most a few MB) on a USB drive (if the partition is larger, the search will take much longer). A file (e.g. `leahcim.text`) with any ASCII text must now be saved on this partition (e.g. "regrebnettuL", `echo "regrebnettuL" >> leahcim.txt`). Since the Linux kernel may not write the files to disk immediately, a manual *filesystem sync* must be performed before continuing with the demo. This can be achieved with `sync -f path/to/drive-mountpoint` or simply by ejecting the drive once. The file can then be deleted `rm leahcim.txt`. For the recovery, the "device" of the demo partition must first be found (simply with `lsblk`. The partition can be located in, e.g. `/dev/sdd1`). Now you can use `sudo strings /dev/sdd1 | grep "regrebnettuL"` to search for the

contents of the deleted file (`strings` gives us all possible strings with a minimum length of 4 that are present on the partition). This works because the filesystem is bypassed via `/dev/<some-drive>`, instead the bytes on the connected drive are viewed directly.

2. `shred` tries to obfuscate the contents of a file by overwriting it multiple times before deletion. However, additional recovery measures of the filesystem or the storage hardware itself may void the deletion attempts of `shred` (see `man shred`)!
3. One would need to use the `--no-preserve-root` flag. (`sudo` is required as well)

1.3 Control & Redirection Operators

In the lecture you have already learned about the control operators `||`, `&&`, `;`, `&`, `()` and `|` (also called "Pipe"), as well as the redirection operators `>`, `>>`, `<` and `<<`. In the following tasks, you will practice using these operators.

1.3.1 Pipes and Output Redirection

On Moodle, in addition to these exercise sheets, you will also find the following two text files: `study1.txt` and `study2.txt` (both files contain a subset of the study courses offered by TUM). For this task, the commands `sort` and `uniq` are helpful (skim the associated `man` pages if needed).

1. What is the difference between `>>` and `>`? Redirect the output of `echo` into different test files and figure it out via trial and error.
2. What is the technical difference between `2>>` and `>>` (resp. `2>` and `>`)?
   (*Hint: Redirect the output of a command that can fail*)
3. Print the contents of both files using the command `cat`.
4. Can the previous invocation of `cat` be optimized to output a large number of files without having to explicitly specify every file? Find out about "globbing" on the Internet.
5. Specify two commands which can be used to output the contents of `study1.txt` and `study2.txt` sorted respectively.
6. Now, in the sorted output of both files, additionally eliminate all duplicates without changing the files.

7. Output in one command the contents of both files sorted and free of duplicates. How could you save this output in a new file named `study-all.txt`?
8. Add the study program "Medizin" to the file `study-all.txt` (without deleting the existing content). Then sort the file again.
9. Now print the first and last 4 lines of `study-all.txt` (The commands `head` and `tail` may be helpful).
10. Let a cow with its tongue sticking out (`" U"`) recite a Haiku using a *Here-Document*.

Solution

1. `>>` appends to a file, `>` truncates the contents of the file first.
2. Only error messages are redirected. The number indicates which file descriptor should be redirected. For "normal" output, the file descriptor 1 is used (which corresponds to *stdout*). Error messages are output via file descriptor 2 (corresponding to *stderr*). A default terminal displays messages from both sources.
3. `cat study1.txt study2.txt`
4. `cat *.txt`, `*.txt` refers to all files ending in `.txt` in the current *CWD*
5. `sort study1.txt` resp. `sort study2.txt`
6. `sort study1.txt | uniq` resp. `sort study2.txt | uniq`
7. `cat study1.txt study2.txt | sort | uniq` and `cat study1.txt study2.txt | sort | uniq > study-all.txt` to save the output to a file.
8. `echo "Medizin" >> study-all.txt`, afterward `sort study-all.txt -o study-all.txt` (**Important:** `sort study-all.txt > study-all.txt` will not work because the file will be *truncated* first!)
9. `cat study-all.txt | head -n 4` and `cat study-all.txt | tail -n 4`
10.
```
cowsay -T " U" <<EOF
Squish, squash, sling that slang.
I'm always right back at ya,
like my... boomerang!
EOF
```

1.3.2 Chaining Commands

In this task commands are (logically) chained. Write a chain of commands that:

1. Change to the folder `test`. Print "`== Error ==`" if the directory change failed.
2. Change to the folder `test`, listing the contents (including hidden files) on a successful directory change.
3. Runs in the background and prints "`TUM`" 3 times while waiting for 5 seconds between each output (you will need the command `sleep`).

Solution

1. `cd test || echo "== Error =="`
2. `cd test && ls -a`
3. `( echo "TUM"; sleep 5; echo "TUM"; sleep 5; echo "TUM" ) &`